BEGINNER CYBERSECURITY WORKSHOP

The Receipts Don’t Lie

Introduction to Log Analysis

```
$ tail -f /var/log/auth.log
Nov  18 08:30:58 session
  started: workshop

Nov  18 09:00:01 user=class
  status=READY

Nov  18 09:00:02 awaiting
input
  ...
_
```

Level: Beginner **Date:** 18 November 2026 **Venue:** UiTM Shah Alam

HACK 2026 CAPTURE THE FLAG ATTACK & DEFENSE

UNIVERSITI TEKNOLOGI MARA

Fakulti Sains Komputer dan Matematik

WELCOME

Before We Begin





Bring a terminal

Linux Terminal or Windows with WSL. Lab files will be provided.



Training Material

Training material will be shared on the Github's link.



Breaks are fixed

We keep to the printed break so every lab gets its full slot.



Ask anytime

This is not a sit-and-listen day. Stop me, ask, and try things on your own machine as we go.

WELCOME

Before We Begin





Training Material

https://github.com/un5eeny0t/iHack2026-The-Receipts-Dont-Lie

TODAY'S PLAN

Workshop Agenda · 0900–1200



09:00 – 09:10	Welcome & Introduction	
09:10 – 10:40	Session 1: Module 1: Understanding Logs Module 2: Reading & Connecting Logs Labs 1: Read Before You Run Labs 2: Your First Terminal Commands Labs 3: SSH Log Analysis and Brute-Force Detection Labs 4: Web Server Log Analysis Labs 5: Mail Server Log (SMTP)	
10:40 – 11:00	Break	BREAK
11:00 – 12:00	Session 2: Lab 6: The Full Picture: Cross-Log Correlation Module 3: From Manual to Automated (SIEM) Recap Q&A	

BY THE END OF TODAY

Learning Objectives

1. Explain what a log is, why it exists, and where logs are stored on Linux and Windows systems

2. Identify key fields in syslog, Apache CLF, auth.log, and Windows Event Log entries

3. Use grep, awk, sort, and uniq in the terminal to manually parse log files

4. Recognize common suspicious patterns: brute-force SSH, scanner User-Agents, and NOQUEUE mail rejects

5. Correlate events across multiple log files to build a simple incident timeline

6. Understand how a SIEM automates what you did manually today, at massive scale



MODULE 1

Understanding Logs

What a log is, why it exists, and where to find one

MODULE 1

What Is a Log?

A log is a **timestamped record of an event**, written automatically by an operating system, application, or device — without a human typing it.

- Generated automatically — the system writes it, you don't
- Append-only by convention — new events are added, old ones are rarely edited
- Plain text on most Linux systems; structured/binary on Windows
- Think of it as the system keeping its own diary of everything that happens

```
$ tail -n 1 /var/log/syslog

Oct  7 09:12:04 labpc01 CRON[1882]:
  (root) CMD (run-parts /etc/cron.hourly)

↑ timestamp   ↑ host   ↑ process[pid]
```

Every log line is built from a few simple parts — once you can name them, you can read any log.

Why Do Logs Exist?





Troubleshooting

Something broke. Logs show what the system was doing right before it happened.



Accountability

Who did what, and when. Useful for audits, change tracking, and disputes.



Security & Forensics

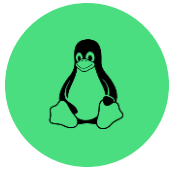
Detect attacks as they happen and reconstruct exactly what an attacker did afterward.



Compliance

Many regulations and industry standards require logs to be kept for a minimum period.

Where Logs Live — Linux



- `/var/log/` is the standard home for almost everything
- **Most files are plain text** — readable with cat, less, grep, no special tools needed
- **Many are rotated automatically** (auth.log, auth.log.1, auth.log.2.gz…)
- Reading them usually just needs **read permission** on the file — some need sudo

```
$ ls -lh /var/log/
drwxr-x---  2 root adm    syslog
-rw-r-----  1 root adm    auth.log
drwxr-xr-x  2 root root   apache2/
-rw-r-----  1 root root   mail.log
```

`/var/log/syslog or /var/log/messages`

General system activity (Debian/Ubuntu vs RHEL/CentOS naming)

`/var/log/auth.log or /var/log/secure`

Authentication: logins, sudo, SSH

`/var/log/apache2/access.log`

Web server requests

`/var/log/mail.log or /var/log/maillog`

Mail server activity

MODULE 1

Where Logs Live — Windows

- Stored as binary .evtx files, not plain text — can’t just cat them
- Viewed through Event Viewer (eventvwr.msc), PowerShell, or wevtutil
- Organized into named logs rather than one big file per topic

Application Events logged by installed software

System Events from Windows components & drivers

Security Logins, logoffs, privilege use — most relevant to this workshop

```
PS> Get-WinEvent -LogName Security -MaxEvents
1

TimeCreated  : 10/7/2026 9:41:08 AM
Id           : 4625
LevelDisplay : Information
ProviderName : Microsoft-Windows-
               Security-Auditing
Message      : An account failed to
               log on.

↑ Event ID 4625 = failed logon
```



QUICK DISCUSSION

A user reports they can’t log in. Which log would you check first — and why?

Turn to the person next to you — 90 seconds, then we’ll compare answers.



MODULE 2

Reading & Connecting Logs

The fields inside syslog, auth.log, Apache CLF, and Windows Event Log

Anatomy of a Syslog Line

```
Oct  7 09:12:04 webserver01 sshd[20413]: Failed password for invalid user admin from 203.0.113.55 port 51422 ssh2
```

Timestamp

Oct 7 09:12:04 — when the event happened

Hostname

webserver01 — which machine logged it

Process[PID]

sshd[20413] — which program, which instance

Message

Free-text description of what happened

Anatomy of auth.log



auth.log uses the same syslog shape, but the message field follows predictable patterns for login-related events:

FAILED LOGIN

```
Failed password for invalid user admin from 203.0.113.55 port 51422 ssh2
```

SUCCESSFUL LOGIN

```
Accepted password for jsmith from 198.51.100.7 port 49210 ssh2
```

PRIVILEGE USE

```
sudo: jsmith : TTY=pts/0 ; PWD=/home/jsmith ; USER=root ;
COMMAND=/bin/systemctl restart nginx
```

Field to watch: the username being tried, the **source IP**, and whether the result was **Failed** or **Accepted**. You'll use exactly these three things in Lab 3.

Anatomy of Apache CLF

```
203.0.113.55 - - [07/Oct/2026:09:50:21 +0800] "GET /wp-login.php HTTP/1.1" 404 1043
"-"
"Mozilla/5.0 (compatible; Nmap Scripting Engine)"
```

Client IP

203.0.113.55 — who made the request

Ident / User

- - — almost always blank on the public web

Timestamp

[07/Oct/2026:09:50:21 +0800]

Request

"GET /wp-login.php HTTP/1.1" — method, path, protocol

Status & size

404 1043 — HTTP status code, bytes sent

User-Agent

Identifies the browser, tool, or script making the request

Anatomy of a Windows Event Log Entry

Event ID

A number that identifies the type of event — the Windows equivalent of a syslog message pattern. e.g. 4624 = successful logon, 4625 = failed logon.

Level

Severity: Information, Warning, Error, or Critical.

Provider / Source

Which component logged the event, e.g. Microsoft-Windows-Security-Auditing.

Logged (Time)

When the event was recorded.

Task Category

A human-readable grouping, e.g. “Logon” or “Process Creation.”

```
PS> Get-WinEvent -LogName Security |
    Where-Object Id -eq 4625 |
    Select-Object TimeCreated,Id

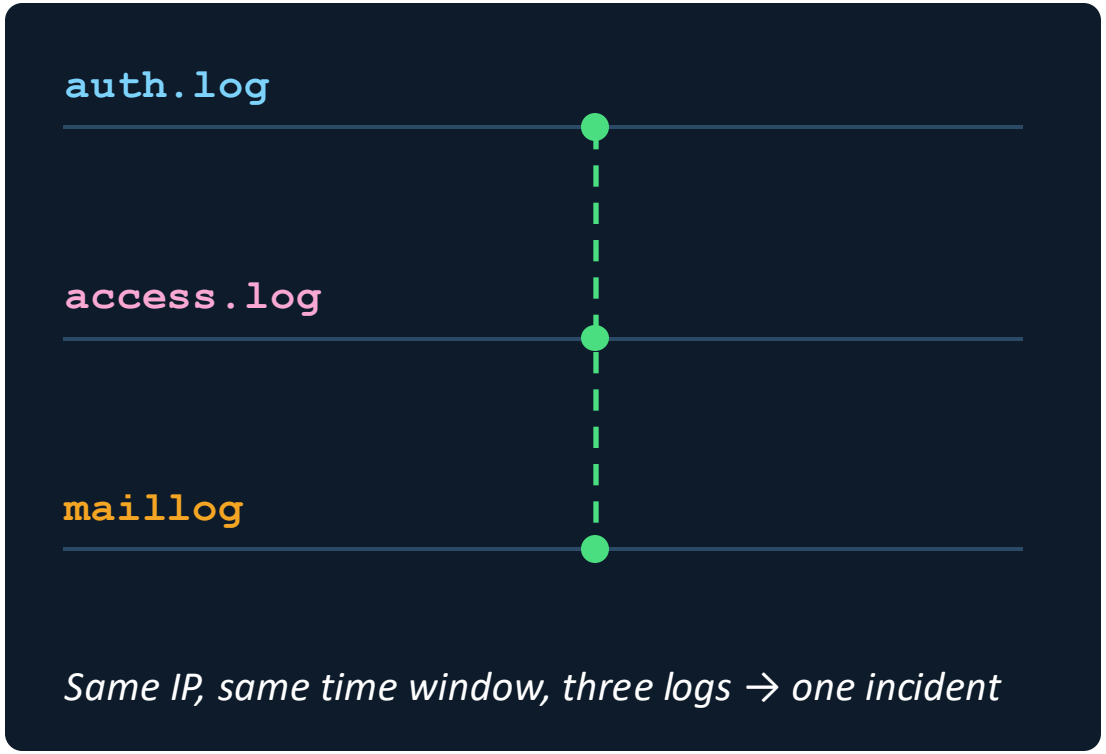
TimeCreated          Id
-----------          --
10/7/2026 9:41:08AM 4625
10/7/2026 9:41:11AM 4625
10/7/2026 9:41:14AM 4625

↑ same pattern as repeated
  "Failed password" lines —
  just a different log format
```

MODULE 2

The Common Thread: Timestamps & Correlation

- Every format we just saw has one thing in common: a timestamp
- Timestamps let you line up events from completely different logs on the same timeline
- Other shared fields — a source IP, a username — act as join keys connecting one log to another
- Clocks must agree: this is why time synchronization (NTP) across servers matters for real investigations





Hands-On Labs

Six labs, one running case: a single attacker, three logs, one story

LAB 1

Read Before You Run

Golden rule: never run a command or script against a system before you understand what every part of it does — especially commands copied from the internet, a chatbot, or a colleague's notes.

- Check the file before touching it: size, type, permissions
- Preview a few lines before processing the whole file
- Know what each flag does — don't copy-paste blind
- On a real system, an unfamiliar log file could itself be evidence — handle it read-only until you know what you're doing

```
$ ls -lh sample-auth.log
-rw-r--r-- 1 lab lab 48K sample-auth.log

$ file sample-auth.log
sample-auth.log: ASCII text

$ head -n 5 sample-auth.log
Oct  7 09:40:02 ... sshd[20410]: ...

$ wc -l sample-auth.log
612 sample-auth.log
```

LAB 2

Your First Terminal Commands

Command	What it does	Example
`grep`	Search for lines matching a pattern	`grep "Failed password" auth.log`
`awk`	Extract or process specific columns	`awk '{print $1, $2, $3}' auth.log`
`sort`	Put lines into order	`sort access.log`
`uniq -c`	Count repeated lines (needs sorted input first)	`sort ips.txt | uniq -c`

The pipe `|` chains commands together — the output of one becomes the input of the next:

```
grep "Failed password" auth.log | awk '{print $11}' | sort | uniq -c | sort -nr
```

LAB 3

SSH Log Analysis



Goal: Spot signs of an SSH brute-force attack in auth.log

```
$ grep "Failed password" sample-auth.log | wc -l
247


247 failed logins in one file — worth a closer look
```

- Start broad: how many “Failed password” lines exist in total?
- Then narrow: which source IP is responsible for most of them?
- Finally, check: did any of those IPs eventually succeed?

Try it yourself

1. `grep "Failed password" sample-auth.log | wc -l`
2. `grep "Accepted password" sample-auth.log`

LAB 3

Spotting the Brute-Force Pattern



```
$ grep "Failed password" sample-auth.log | awk '{print $(NF-3)}' \
  | sort | uniq -c | sort -nr | head -5

    203  203.0.113.55
      9  198.51.100.30
      6  198.51.100.31
```

- One IP, hundreds of attempts
- Many different usernames tried
- Tight, regular time intervals
- An “Accepted password” right after — compromise

203.0.113.55 is now your suspect IP — carry it forward into the next labs.

LAB 4

Web Server Log Analysis

Goal: find scanner / attack traffic in access.log — starting with our suspect IP, 203.0.113.55

Giveaways to look for

- Scanner / tool User-Agents: Nmap, sqlmap, python-requests, curl, masscan, Go-http-client
- Repeated requests to sensitive paths: /wp-login.php, /phpmyadmin, /.env, /admin
- A wall of 404s from one IP — it's probing for things that don't exist

```
$ grep 203.0.113.55 sample-access.log \
  | awk -F'"' '{print $6}' | sort | uniq -c

   41  Nmap Scripting Engine
   18  sqlmap/1.7

$ grep 203.0.113.55 sample-access.log \
  | grep -Eo "GET [^ ]+" | sort | uniq -c \
  | sort -nr | head -3

   22  GET /wp-login.php
   15  GET /phpmyadmin/
   12  GET /.env
```

LAB 5

Mail Server Log Analysis

Goal: Understand rejected mail and spot abuse in maillog

```
Oct  7 10:15:02 mailsrv postfix/smtpd[18233]:
NOQUEUE: reject: RCPT from unknown[198.51.100.23]: 554 5.7.1
<spam@evil.example>: Relay access denied; from=<forged@example.com>
 to=<victim@uitm.edu.my>
```

NOQUEUE: reject

Postfix refused this message during the SMTP conversation, before it was ever accepted into the delivery queue — most often because of relay restrictions, a blocklist hit, or an invalid recipient.

Try it yourself

```
grep "NOQUEUE: reject" sample-maillog \
  | wc -l
grep "NOQUEUE: reject" sample-maillog \
  | awk -F"from=" '{print $2}' \
  | sort | uniq -c | sort -nr
```

LAB 6

The Full Picture: Cross-Log Correlation



Goal: Search all three logs for one IP, and line up what you find by time

```
$ grep 203.0.113.55 sample-auth.log sample-access.log sample-mail.log
  | sort -k1
```

- One grep, three filenames — searches all of them at once
- grep prefixes each match with its source filename automatically
- Sort the combined output by timestamp to see the order events happened

LAB 6

Building the Incident Timeline

09:42	Dozens of failed SSH logins begin	`auth.log`
09:58	An “Accepted password” — the attacker is in	`auth.log`
10:05	Requests to /wp-login.php from the same IP	`access.log`
10:15	An outbound relay attempt is rejected	`mail.log`

One IP. Three logs. One story — and you built it by hand.



From Manual to Automated

What you just did by hand is exactly what a SIEM does at scale

WRAP-UP

What a SIEM Does



SIEM = Security Information and Event Management

01



Centralize

Collects logs from every server, firewall, and endpoint into one place

02



Normalize

Converts different formats (syslog, CLF, Event Log…) into one common shape

03



Correlate

Automatically links related events across sources — in real time

04



Alert

Notifies an analyst the moment a known bad pattern is matched

You did this by hand, for one IP, across three files, in about half an hour. **A SIEM does it for thousands of hosts, every second.**

WRAP-UP

FROM MANUAL PARSING TO SIEM



Splunk

Enterprise SIEM. SPL query language.
High adoption in large corporations.

Elastic Security

Open-source SIEM built on Elasticsearch.
KQL / ES|QL query language.
Common in Malaysian enterprise & gov.

Microsoft Sentinel

Cloud-native SIEM on Azure.
KQL queries. Integrates with M365.

Wazuh

Open source. Great for learning.
Free VM available. Start here.

RECAP

Today's Objectives — Revisited

- Explain what a log is, why it exists, and where logs live on Linux and Windows
- Identify key fields in syslog, Apache CLF, auth.log, and Windows Event Log
- Use grep, awk, sort, and uniq to manually parse log files
- Recognize brute-force SSH, scanner User-Agents, and NOQUEUE mail rejects
- Correlate events across multiple logs into a simple incident timeline
- Understand how a SIEM automates this manual work at massive scale



CLOSING THOUGHT

The Receipts Don’t Lie.

Every action on a system leaves a trace. The skill isn’t making the trace appear — it’s knowing how to read it.

```
$ echo "You can read the receipts now."
You can read the receipts now.
```



Questions & Thank You

The Receipts Don’t Lie: Introduction to Log Analysis

Trainer: **Syakir Ameer Roslan** **Email:** **syakir@eclogic.io**
Trainer: **Abd Halim Bin Mahmood** **Email:** **halim@eclogic.io**

```
$ history | tail -1
Read the logs. Trust the receipts. Stay curious.
_
```